

18

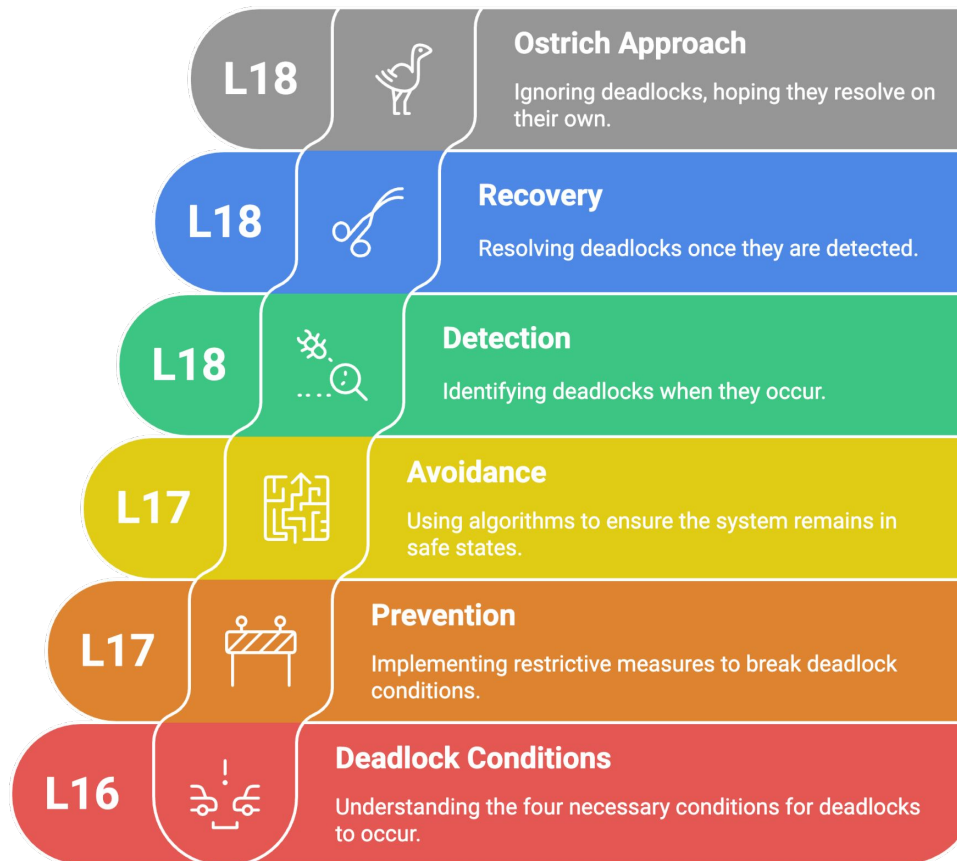
Lecture 18

Deadlocks III: Detection, Recovery, and Ignorance

by - Brijesh Shukla
C07: Operating Systems

THE STORY SO FAR

The Question: What if we can't prevent or avoid deadlocks?



THE LIMITATION OF AVOIDANCE

Banker's Algorithm Requires:

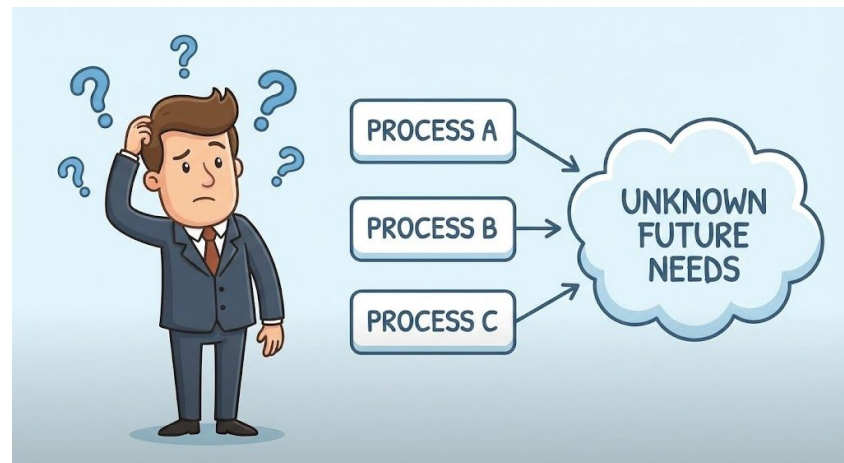
- Processes declare MAXIMUM resource needs in advance
- Static set of processes
- Known resource types

Reality Check - Modern Systems:

- ✗ Web servers don't know future memory needs
- ✗ Text editors don't know how many files you'll open
- ✗ Compilers don't know how big your code will get
- ✗ Dynamic process creation/termination
- ✗ Unpredictable user behavior

The Problem: We can't predict the future!

Solution: Detection - work with what we HAVE, not what we MIGHT need



The Frozen Mac Scenario

Your Mac has frozen with the spinning beach ball.
You've waited 5 minutes. Nothing happens.

What do you do?

- A) Wait longer, hoping it resolves itself
- B) Force quit the application
- C) Restart your Mac entirely
- D) Throw it out the window



This connects to TODAY's topic: Detection and Recovery!

DETECTION PHILOSOPHY

Prevention/Avoidance:

- Be overly cautious
- Restrict behavior to avoid problems
- Low resource utilization
- "Never let bad things happen"

Detection & Recovery:

- Allow risky behavior
- Monitor for problems
- High resource utilization
- "Let bad things happen, then fix them"

The Trade-off:

- ✓ Better resource usage (fewer restrictions)
- ✓ Don't need to predict future
- ✗ Recovery overhead when deadlock occurs
- ✗ Potential data loss during recovery



DETECTION ALGORITHM OVERVIEW

The Detection Approach:

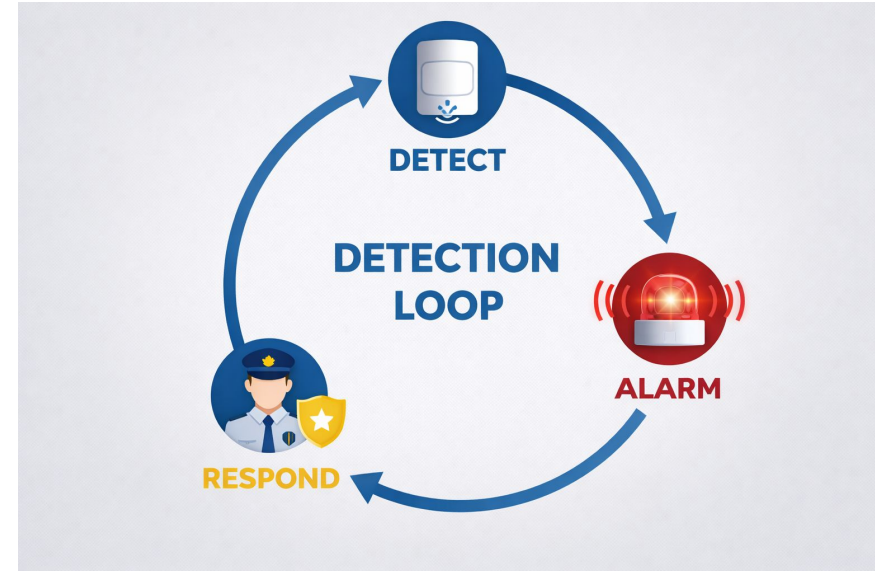
1. Let processes run freely (no restrictions)
2. Periodically check: "Is there a deadlock RIGHT NOW?"
3. If deadlock found: Execute recovery strategy
4. If no deadlock: Continue normal operation

Key Insight: We examine CURRENT state, not future possibilities

Detection Algorithm Checks:

- Who holds which resources? (Allocation)
- Who is requesting which resources? (Request)
- What's currently available? (Available)
- Can we find an execution order where everyone finishes?

If NO valid execution order exists → **DEADLOCK!**



DATA STRUCTURES

Data Structures (Similar to Banker's, but Current State!):

Available[m]: Resources currently available

- m = number of resource types
- Example: Available = [2, 1, 0] means 2 of R1, 1 of R2, 0 of R3

Allocation[n][m]: Resources currently held by each process

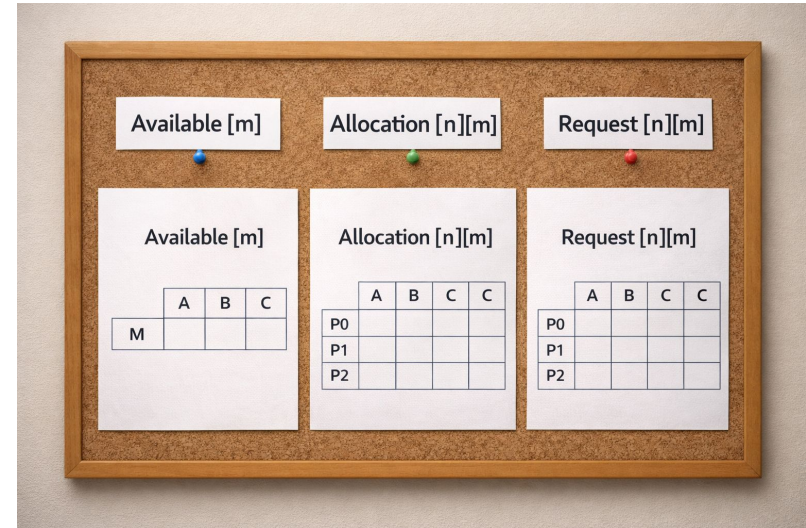
- n = number of processes
- Allocation[n][m] = resources of type m held by process n

Request[n][m]: Current outstanding resource requests

- NOT maximum needs (that's Banker's!)
- What each process is asking for RIGHT NOW

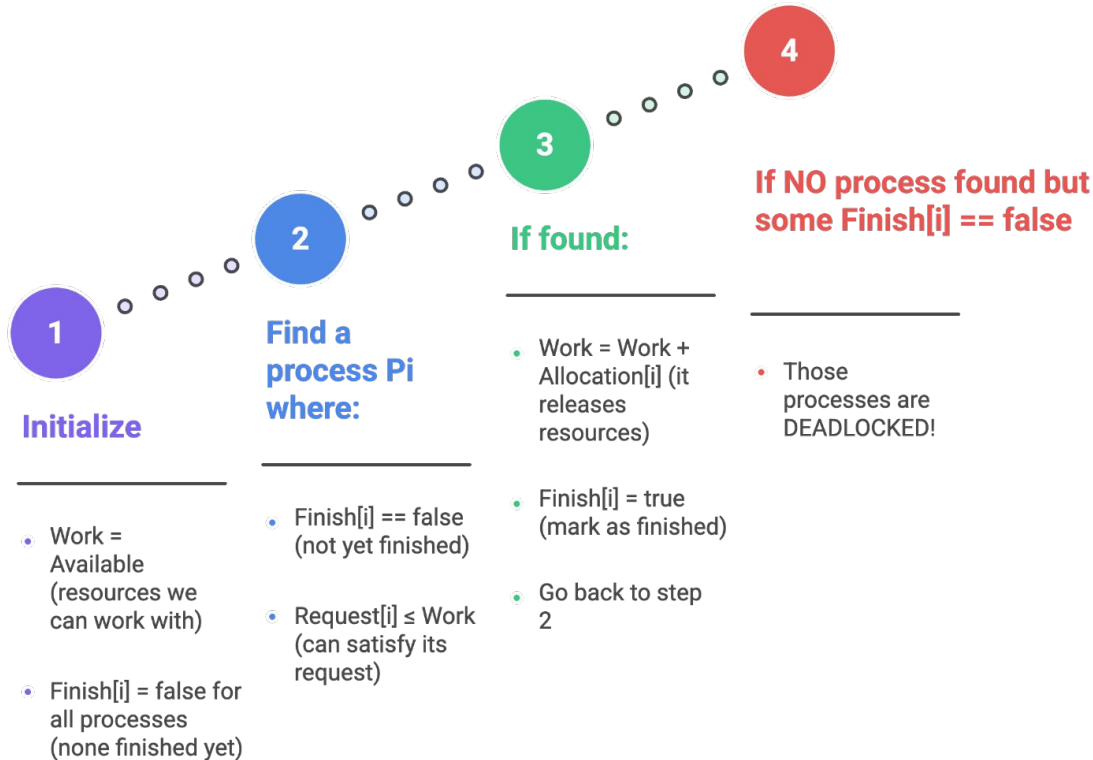
Key Difference from Banker's:

Banker's uses Max needs → Detection uses Current requests



THE PROCESS

Detection Algorithm Steps



Intuition: Try to find an order where processes can finish.
If we can't finish everyone, remainder is deadlocked.

DETECTION EXAMPLE

Example System:

Available = [0, 0, 0]

Process	Allocation (A B C)	Request (A B C)
P0	0 1 0	0 0 0
P1	2 0 0	2 0 2
P2	3 0 3	0 0 0
P3	2 1 1	1 0 0
P4	0 0 2	0 0 2

Result: All Finish = true
→ NO DEADLOCK! ✓

Trace: Work = [0,0,0], all Finish = false

Request [0,0,0] ≤ [0,0,0]?

YES. Work = [0,1,0], Finish[0] = true

Request [0,0,0] ≤ [0,1,0]?

YES. Work = [3,1,3], Finish[2] = true

Request [1,0,0] ≤ [3,1,3]?

YES. Work = [5,2,4], Finish[3] = true

Request [2,0,2] ≤ [5,2,4]?

YES. Work = [7,2,4], Finish[1] = true

Request [2,0,2] ≤ [5,2,4]?

YES. Work = [7,2,4], Finish[1] = true

Question: Is there a deadlock?

Let's trace through the algorithm...



Detection Understanding Check

What if P1's request was [2, 2, 2] instead of [2, 0, 2]?

Original: P1 requests [2, 0, 2]

Modified: P1 requests [2, 2, 2]

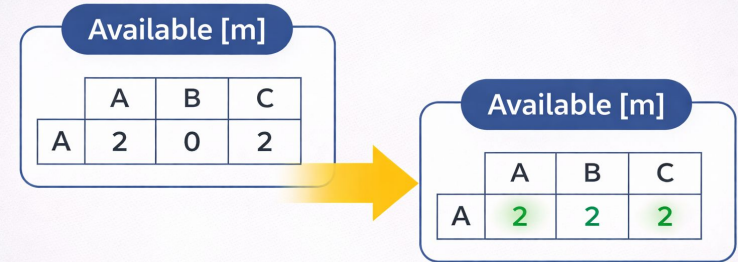
Would there be a deadlock?

- A) No, same result
- B) Yes, P1 would be deadlocked
- C) Yes, all processes would be deadlocked
- D) Cannot determine

ANSWER: A) No, same result

In Banker's Algorithm, if P1 requests [2, 2, 2] and granting it makes the system unsafe, the request is simply denied and P1 waits. Since unsafe allocation is never allowed, no deadlock occurs.

AVAILABLE VECTOR CHANGE



WAIT-FOR GRAPHS (WFG)

Problem: Full detection algorithm is complex for simple cases

Simplification: Wait-For Graph (WFG)

When to Use:

- ✓ Each resource type has only ONE instance
- ✓ Common in many systems (mutex locks, files)

WFG Structure:

- Nodes: Processes ONLY (no resource nodes!)
- Edge $P_i \rightarrow P_j$: " P_i is waiting for a resource held by P_j "

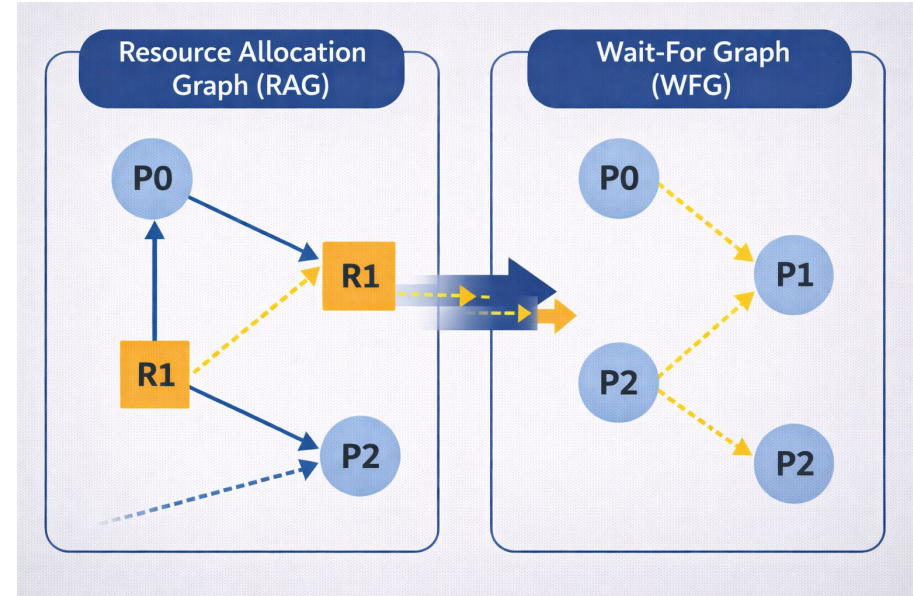
Deadlock Detection:

Deadlock exists $\Leftrightarrow$ Cycle exists in WFG

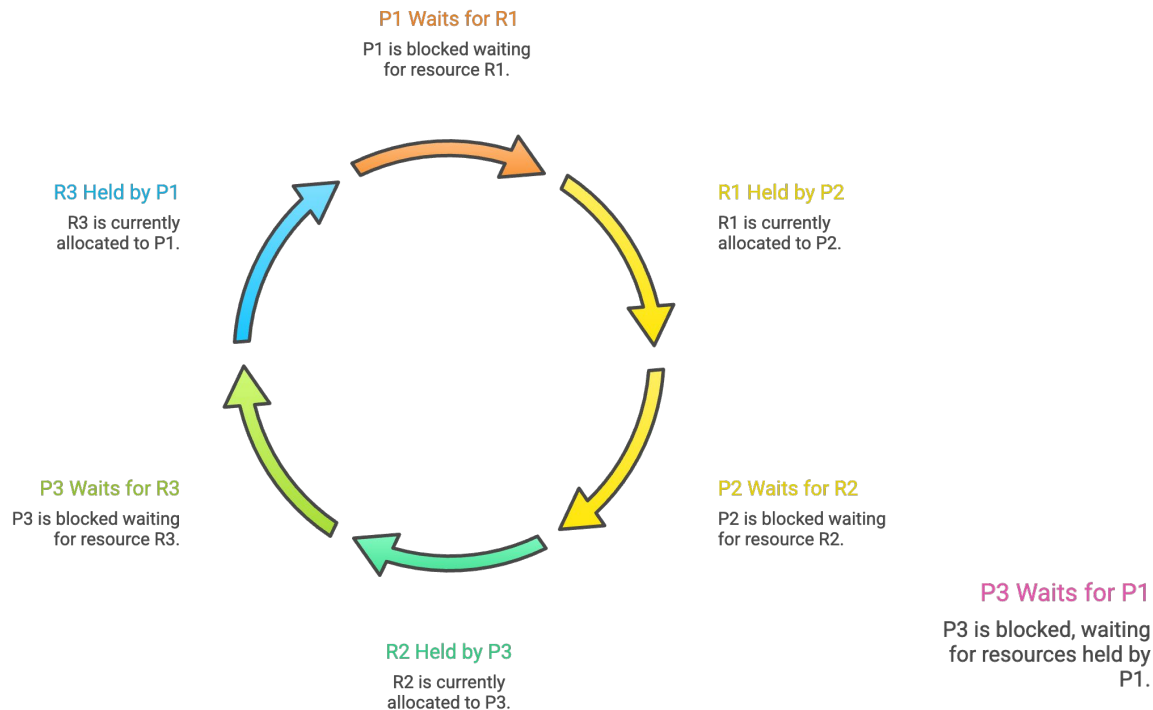
Converting RAG to WFG:

If RAG has: $P_i \rightarrow R_j \rightarrow P_k$

Then WFG has: $P_i \rightarrow P_k$ (remove resource node)



WFG EXAMPLE

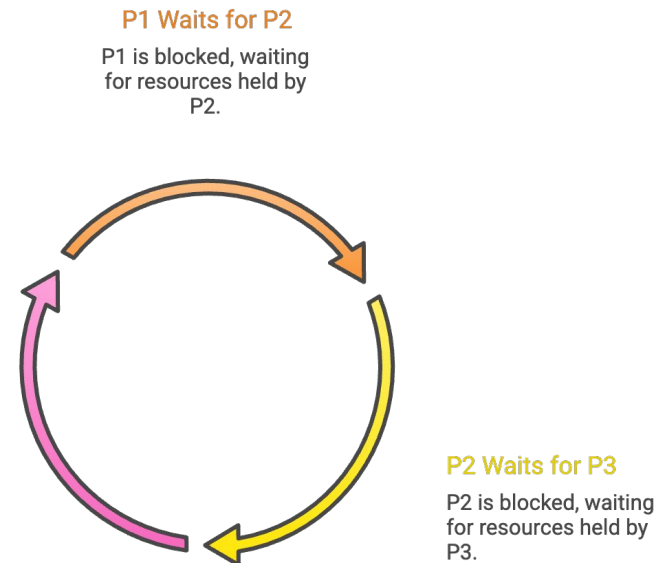


Result: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P1$

CYCLE DETECTED! ⚠ DEADLOCK!

The cycle: All three processes waiting in a circle

Converted to WFG



Find the Deadlock!

Given this Wait-For Graph:

P1 $\rightarrow$ P2

P2 $\rightarrow$ P3

P3 $\rightarrow$ P4

P4 $\rightarrow$ P2

P5 $\rightarrow$ P6

Tasks:

1. Identify all cycles in this graph
2. Which processes are deadlocked?
3. Which processes are NOT deadlocked?
4. If you had to terminate ONE process to break the deadlock, which would you choose?

Solution:

1. Cycle Found: P2 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P2

2. Deadlocked Processes: P2, P3, P4
These three are stuck in a circular wait!

3. NOT Deadlocked: P1, P5, P6

- P1 is waiting for P2 (but not in the cycle)
- P5 $\rightarrow$ P6 is a linear chain (no cycle)

4. Best Process to Terminate:
ANY of P2, P3, or P4 breaks the cycle!

- Terminating P2: Breaks P2 $\rightarrow$ P3, frees P1
- Terminating P3: Breaks P3 $\rightarrow$ P4
- Terminating P4: Breaks P4 $\rightarrow$ P2

Choose based on victim selection criteria (coming next!)

You Found the Deadlock. Now What?

Once Deadlock is Detected, We Must Break It!

Two Main Strategies:

Strategy 1: Process Termination

- Kill one or more processes
- Breaks the circular wait
- Simple but wasteful (loses work)

Strategy 2: Resource Preemption

- Forcibly take resources from processes
- Give them to others
- Requires checkpointing/rollback

The Victim Selection Problem:

Which process should suffer?

This is a policy decision, not a technical one!

Let's explore both strategies...



PROCESS TERMINATION

Process Termination Options:

Option A: Abort ALL Deadlocked Processes

- ✓ Simple - guarantees deadlock resolution
- ✓ No need to select victims
- ✗ Extremely expensive (loses ALL progress)
- ✗ Drastic - like burning down house to kill spider

Option B: Abort ONE Process at a Time

- ✓ Minimizes loss (kill only what's needed)
- ✓ May only need to kill one
- ✗ Expensive - must re-run detection after each kill
- ✗ Iterative process

Victim Selection Criteria (Who dies?):

1. Priority - Kill lower-priority first
2. Compute time - How much work invested?
3. Resources held - How many resources freed?
4. Resources still needed - Close to completion?
5. Type - Interactive vs batch (batch easier to restart)



VICTIM SELECTION EXAMPLE

Deadlocked Processes

Process	Priority	Runtime	Resources	Completion
P1	10	45 min	5	90%
P2	5	5 min	3	10%
P3	8	20 min	4	50%
P4	3	60 min	6	95%

There's no perfect answer - it's a trade-off!

Selection Criteria:

By Priority: Kill P4 (priority = 3, lowest)

By Compute Time: Kill P2 (only 5 min invested)

By Resources: Kill P4 (frees 6 resources)

By Completion: Kill P2 (only 10% done, least wasted)

The Dilemma:

P4 has lowest priority BUT 60 minutes invested and 95% done!

P2 has only 5 minutes invested BUT higher priority!

macOS FORCE QUIT

When your Mac freezes:

⌘ + ⌥ + Esc → Force Quit Applications

What's happening?

1. Detection: You (the user) detected something is wrong
2. Selection: You choose which app to terminate
3. Recovery: OS terminates the process
4. Cleanup: OS reclaims resources

This is Manual Recovery!

- User acts as the detection algorithm
- User applies victim selection criteria
- OS provides the termination mechanism

Modern macOS:

- Watchdog timers for hung processes
- Automatic responder termination
- User choice for application termination



RESOURCE PREEMPTION

Resource Preemption Approach:

Instead of killing process, take its resources!

Steps:

1. Select a Victim - Which process to preempt?
2. Rollback - Return process to safe earlier state
3. Reassign - Give resources to waiting process
4. Resume - Let victim try again later

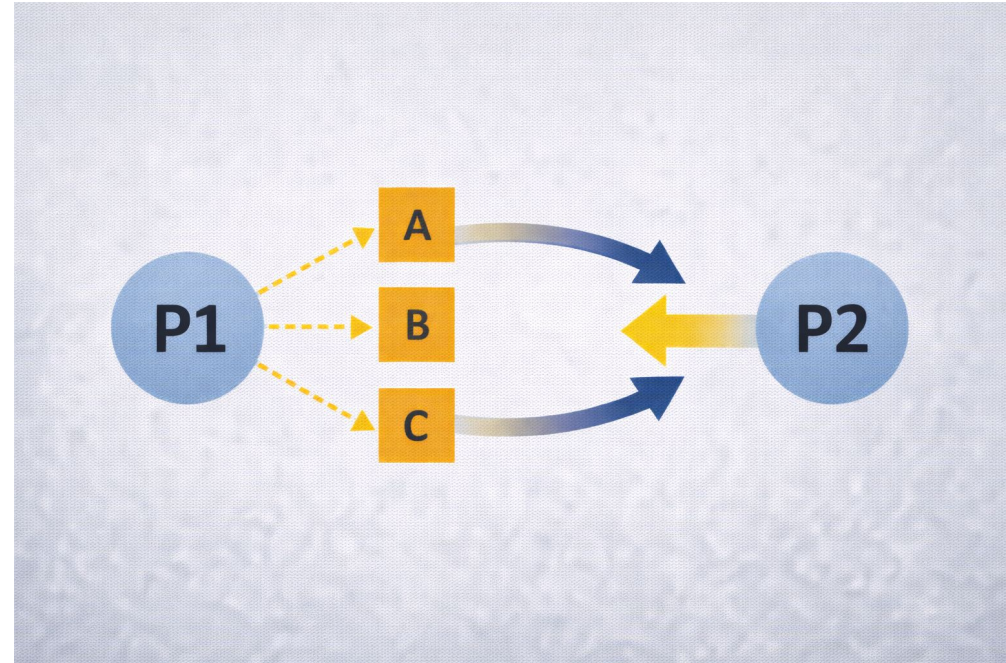
Rollback Options:

Total Rollback:

- Abort and restart from beginning
- Simple but wasteful
- Like rebooting your computer

Partial Rollback:

- Return to a saved checkpoint
- Requires proactive checkpointing
- Like restoring from a save point in a game



The Rollback Mechanism

Checkpointing = Saving Process State:

What to Save:

- Memory contents (heap, stack)
- CPU registers (PC, SP, etc.)
- Open files and positions
- Resource allocations

When to Checkpoint:

- Periodically (every N minutes)
- Before critical operations
- At transaction boundaries

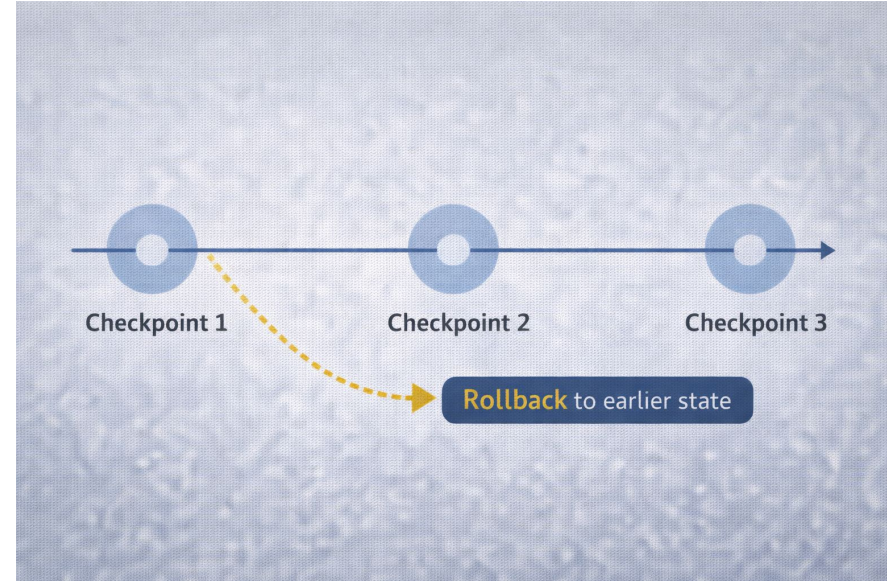
Challenges:

Overhead:

- ✓ Storage space (snapshots are large!)
- ✓ Time to save state (pause process)
- ✓ I/O to disk

Preemption Limitations:

- ✓ Not all resources can be preempted!
- **Memory:** ✓ Can swap to disk
- **CPU time:** ✓ Can reschedule
- **Printer mid-job:** ✗ Can't undo printed pages
- **Network connections:** ✗ Can't roll back sent data



RECOVERY SIMULATION

Deadlock detected among P1, P2, P3, P4:

Process	Priority	Runtime	Resources	Completion
P1	10	45 min	5	90%
P2	5	5 min	3	10%
P3	8	20 min	4	50%
P4	3	60 min	6	95%

Questions:

1. Using priority-based selection, which terminate?
2. Using compute-time-based selection, which?
3. If you need to free maximum resources quickly?
4. What if P4 is an interactive user's application and P2 is a batch job?
5. What's your final recommendation and why?

Key Insight: NO PERFECT ANSWER!

System designers must prioritize what matters: fairness, efficiency, user experience?

Analysis:

1. Priority-Based: Kill P4 (priority = 3)

⚠ But P4 is 95% done with 60 min invested - HUGE waste!

2. Compute-Time: Kill P2 (only 5 min invested)

✓ Minimal wasted work

⚠ But P2 has higher priority (5 vs 3)

3. Maximum Resources: Kill P4 (holds 6)

✓ Frees most resources immediately

⚠ Same problem - 95% done, 60 min wasted

4. Interactive vs Batch: Kill P2 (batch)

✓ Batch jobs easier to restart

✓ User experience preserved

5. Best Choice: Probably P2

- Least work wasted (5 min)
- Batch job (easy to restart)
- Only 10% complete
- Trade-off: Slightly higher priority

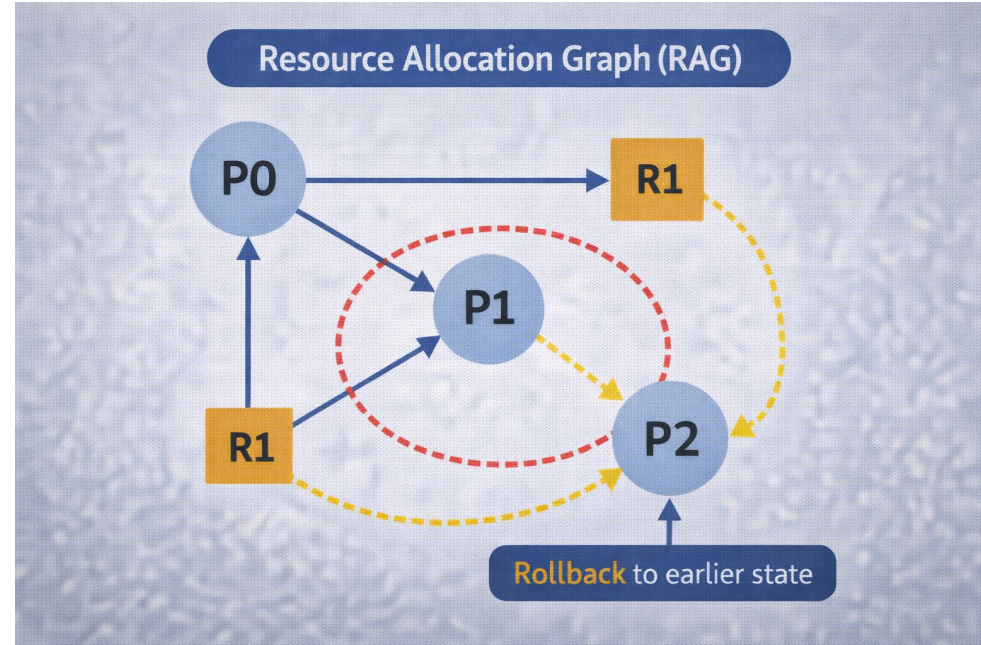
Recovery Understanding Check

What does a cycle in a Wait-For Graph indicate?

- A) The system is in a safe state
- B) Deadlock is possible in the future
- C) Deadlock currently exists
- D) Resources are being wasted

ANSWER: C) Deadlock currently exists

A cycle in a WFG means processes are currently waiting in a circle - deadlock exists RIGHT NOW. This is different from avoidance (which checks for future possibilities) or RAGs with multiple instances (where cycles don't always mean deadlock).



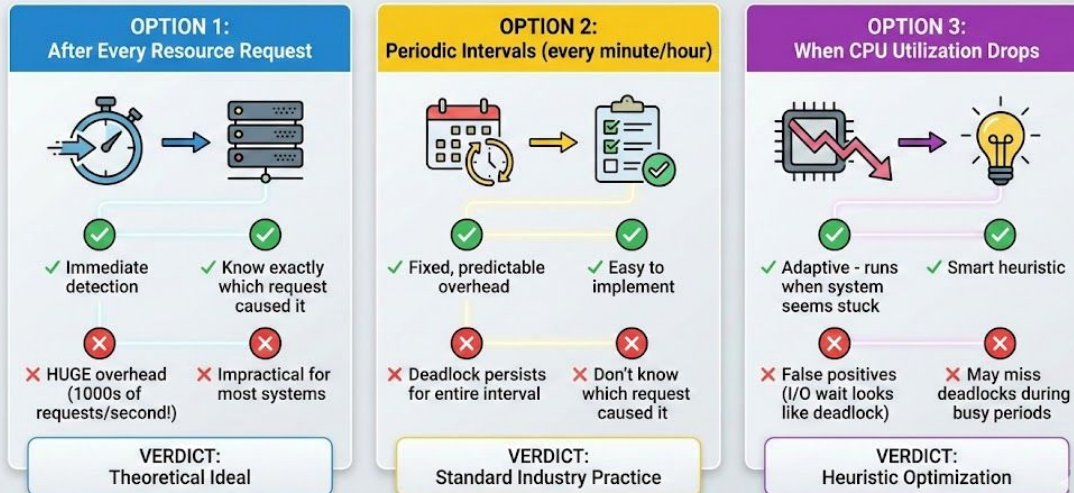
DETECTION FREQUENCY

Detection is NOT Free!

Costs:

- CPU time to run algorithm: $O(m \times n^2)$
- Memory for data structures
- System disruption during check

WHEN TO INVOKE DEADLOCK DETECTION?



DETECTION OVERHEAD CALCULATION

Example Calculation:

Assumptions:

- Detection algorithm takes 10ms
- System handles 10,000 requests/second

Strategy A: Detect every 100 requests

- Detections per second: $10,000 / 100 = 100$
- Overhead: $100 \times 10\text{ms} = 1000\text{ms} = 1 \text{ second}$
- 10% CPU spent on detection! ❌

Strategy B: Detect every 1000 requests

- Detections per second: $10,000 / 1000 = 10$
- Overhead: $10 \times 10\text{ms} = 100\text{ms}$
- 1% CPU spent on detection ✓

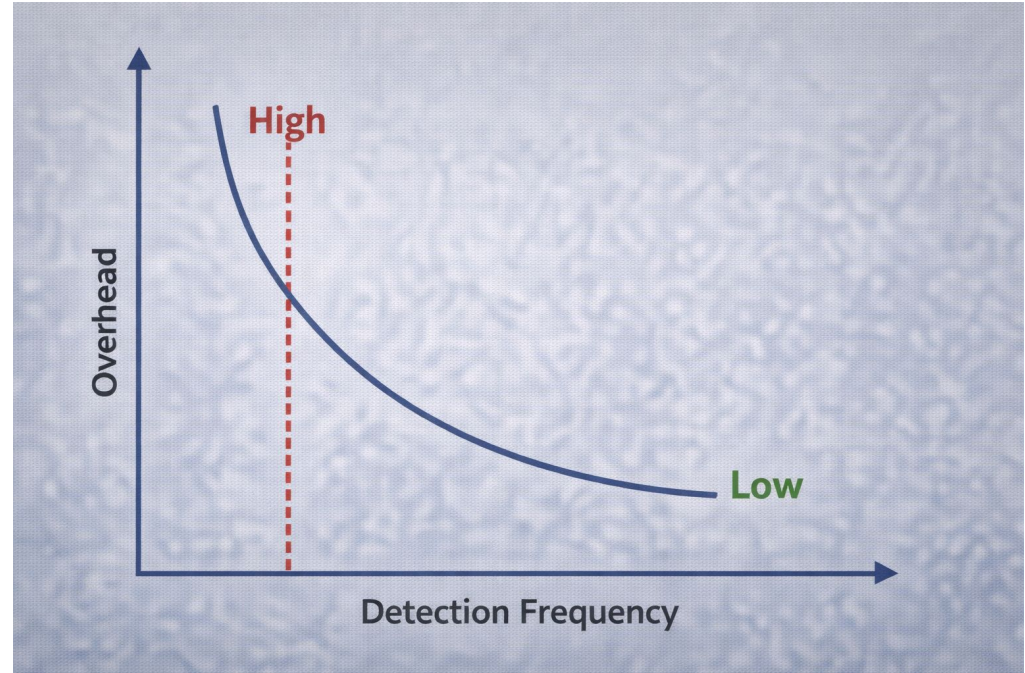
Strategy C: Detect every 60 seconds

- Overhead: $10\text{ms} / 60 \text{ seconds} \approx 0.017\%$ ✓✓
- But deadlock could last up to 60 seconds! ❌

The Balance:

More frequent = Shorter deadlock duration but higher overhead

Less frequent = Lower overhead but longer deadlock duration



Detection Frequency Check

Why is detecting deadlock after every resource request impractical?

- A) It doesn't work correctly
- B) It causes deadlocks itself
- C) The overhead is too high for high-request systems
- D) It's impossible to implement

ANSWER: C) The overhead is too high for high-request systems

Running the detection algorithm ($O(m \times n^2)$) after every request would consume enormous CPU time when systems handle thousands of requests per second, making it impractical despite finding deadlocks immediately.

THE OSTRICH ALGORITHM

The Ostrich Algorithm:

Philosophy: "Stick your head in the sand"

Strategy:

1. Ignore deadlocks completely
2. If system hangs → User reboots
3. Pretend the problem doesn't exist

Wait, WHAT?! Is this serious?

YES! Many systems use this approach!

When Is This Reasonable?

- ✓ Deadlocks are extremely rare (once per year)
- ✓ Cost of prevention/detection > cost of occasional reboot
- ✓ Non-critical systems (personal computers)
- ✓ Users know how to reboot

Real-World Usage:

- UNIX/Linux: Historically for user-level deadlocks
- Early Windows: Relied on user reboots
- macOS: Hybrid - some detection, but users often force quit



OSTRICH JUSTIFICATION

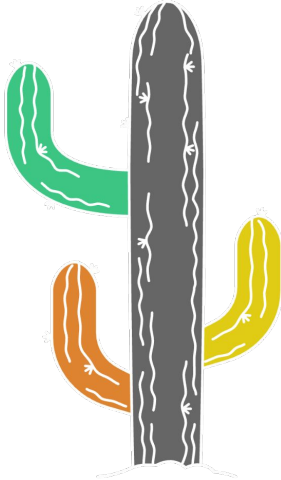
Cost-Benefit Analysis

Prevention/Avoidance

- Implementation complexity: HIGH
- Runtime overhead: MEDIUM to HIGH
- Resource utilization: LOW (restrictive)
- Perfect correctness: YES

Detection & Recovery

- Implementation complexity: HIGH
- Runtime overhead: VARIABLE
- Recovery cost: HIGH (data loss)
- Perfect correctness: EVENTUALLY



Ostrich (Ignorance)

- Implementation complexity: ZERO
- Runtime overhead: ZERO
- Handling cost: User time + reboot
- Perfect correctness: NO

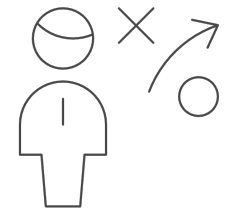
Annual Cost



Prevention

365 days

Annual cost of prevention



Ostrich

5 minutes

Annual cost of 1 reboot

WHEN OSTRICH FAILS

Unacceptable for Ostrich:

✗ Safety-Critical Systems

- Medical devices (pacemakers, surgery robots)
- Aircraft control systems
- Nuclear power plants
- Automotive systems (self-driving cars)

✗ Mission-Critical Systems

- Financial trading systems (lost money!)
- Air traffic control
- Emergency services (911 dispatch)
- Data centers (cloud services)

✗ High-Value Work

- Video editing (hours of unsaved work)
- Software development (lost code)
- Scientific simulations (days of computation)

These systems MUST use prevention/avoidance/detection!



The Consequences:

- Data loss → Unacceptable
- Safety risk → Catastrophic
- Financial loss → Expensive
- User trust → Damaged

All Deadlock Strategies

Strategy Comparison

Strategy	 Prevention	 Avoidance	 Detection+Rec	 Ostrich
Resource Use	Low	Medium-High	High	High
Complexity	Low	High	Med-High	Minimal
Overhead	Low	Medium	Variable	Minimal
Best For	Safety-critical	Known max needs	Dynamic workloads	Rare deadlocks

Real-World Hybrid Approaches:

macOS Strategy:

- ✓ Resource ordering for kernel locks (prevention)
- ✓ Timeouts and watchdogs (pseudo-detection)
- ✓ User force quit (manual recovery)
- ✓ Grand Central Dispatch avoids many deadlocks (design)

Database Systems:

- ✓ Timeout-based detection
- ✓ Transaction rollback (recovery via preemption)
- ✓ Two-phase locking (prevention)

Embedded/Real-Time:

- ✓ Prevention (resource ordering, no hold-and-wait)
- ✓ Cannot afford detection or recovery complexity

Key Takeaways

What We've Learned:

1. Detection Finds Current Deadlocks

- Use RAG/WFG analysis and cycle detection
- Check: "Can all processes finish with current resources?"

2. Recovery Requires Hard Choices

- Terminate processes (simple but wasteful)
- Preempt resources (needs checkpointing)
- Victim selection is policy, not algorithm

3. Frequency Matters

- Trade-off: overhead vs deadlock duration
- Most systems use timeouts/heuristics

4. Ostrich is Sometimes Right

- Ignoring rare problems saves resources
- Acceptable for non-critical consumer systems

5. Real Systems Use Hybrid Approaches

- Mix prevention, detection, and ignorance
- Context determines strategy

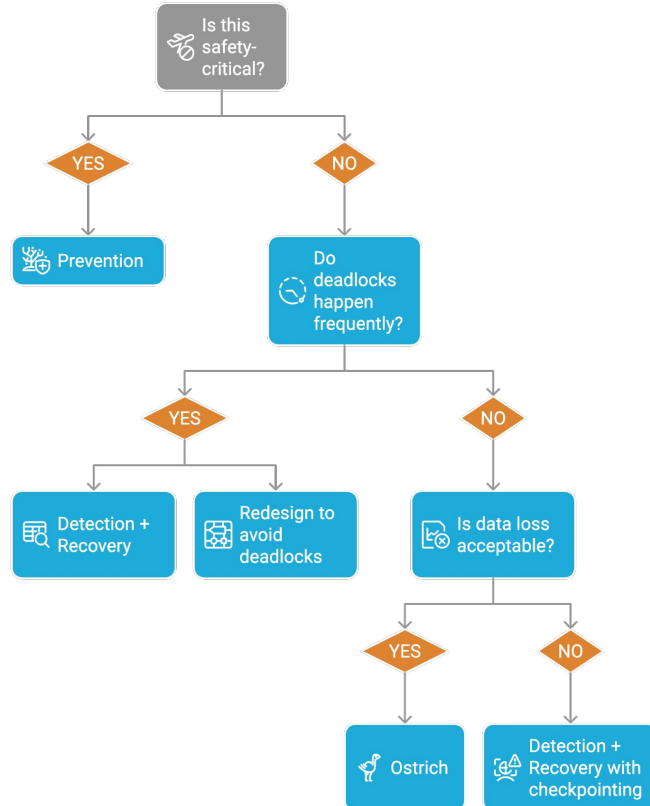


The Engineer's Question:

"How often do deadlocks occur, and what's the cost if they do?"

Practical Guidelines

Deadlock Strategy Selection



Special Cases:

- Known max needs + static processes → Avoidance (Banker's)
- High-performance systems → Prevention (lowest overhead)
- User-facing applications → Timeouts + manual recovery

The Reality Check:

Most modern systems use a MIX:

- Prevention for kernel resources
- Detection/timeout for user resources
- Ostrich for rare edge cases

Thank You!

Thank you for your attention. Feel free to ask questions or revisit the topics covered.